CS 301
High-Performance Computing

Assignment 06 & 07

Parallel Interpolation & Particle Mover

Visha Sitapara (202301414)
Dhruv Patel (202301095)

April 19, 2026

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 Hardware Details for LAB207 PCs . . . 3
2.2 Hardware Details for HPC Cluster (Node gics0) . . . 4

3 Problem Description **5**
3.1 Linear Interpolation (Scatter) . . . 5
3.2 Particle Update using Reverse Interpolation (Mover) . . . 5
3.3 Parallelization Strategies: . . . 6

4 Benchmarking Methodology **6**

5 Graphical Results **7**
5.1 Assignment 06: Scatter Interpolation Results . . . 7
5.1.1 Speedup vs Cores . . . 7
5.1.2 Execution time vs Cores . . . 12
5.2 Assignment 07: Particle Mover Results . . . 17
5.2.1 Speedup vs Cores . . . 17
5.2.2 Execution time vs Cores . . . 20

6 Analysis and Questions **23**
6.1 Assignment 06 Analysis (Scatter Interpolation) . . . 23
6.2 Assignment 07 Analysis (Particle Mover) . . . 27

7 Conclusions **32**

1 Introduction

This lab focuses on the parallelization of the complete particle-mesh workflow, including both the scatter point-to-mesh interpolation and the reverse interpolation (gather: mesh-to-point) acting as a particle mover. It builds upon a serial implementation from Assignment 03 and aims to improve computational performance using OpenMP while maintaining correctness. The experiment highlights the contrasting challenges of handling race conditions during the scatter phase versus optimizing read-heavy, memory-bound operations during the gather phase, while maintaining correct positional updates for particles across multiple iterations. In addition, it evaluates scalability and efficiency across different problem sizes and thread configurations.

2 Hardware Details

2.1 Hardware Details for LAB207 PCs

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Address sizes: 46 bits physical, 48 bits virtual
- Byte Order: Little Endian
- CPU(s): 12
- On-line CPU(s) list: 0-11
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 151
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- Stepping: 5
- CPU max MHz: 4600.0000
- CPU min MHz: 800.0000
- BogoMIPS: 5990.40
- Virtualization: VT-x

- L1d cache: 288K
- L1i cache: 192K
- L2 cache: 7.5M
- L3 cache: 18M
- NUMA node0 CPU(s): 0-11
- Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc aperfmperf eagerfpu pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm epb invpcid_single tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts

2.2 Hardware Details for HPC Cluster (Node gics0)

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- Byte Order: Little Endian
- CPU(s): 24
- On-line CPU(s) list: 0-23
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 2
- NUMA node(s): 2
- Vendor ID: GenuineIntel
- CPU family: 6
- Model: 63
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- Stepping: 2
- CPU MHz: 2907.656
- BogoMIPS: 4804.69
- Virtualization: VT-x

- L1d cache: 32K
- L1i cache: 32K
- L2 cache: 256K
- L3 cache: 15360K
- NUMA node0 CPU(s): 0-5, 12-17
- NUMA node1 CPU(s): 6-11, 18-23

3 Problem Description

3.1 Linear Interpolation (Scatter)

Given a set of scattered points in a 2D domain with known function values:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \ldots, N\}$$

where (x_i, y_i) represent spatial coordinates and f_i are known function values. For this assignment, assume $f_i = 1$. The objective is to interpolate these scattered values onto a structured mesh grid of size $M \times M$. The structured mesh consists of regularly spaced grid points:

$$G = \{(X_i, Y_j) \mid X_i = i\Delta x,\ Y_j = j\Delta y,\ i, j = 0, 1, \ldots, M-1\}$$

where Δx and Δy define uniform grid spacing:

$$\Delta x = \frac{X_{\max}}{M}, \quad \Delta y = \frac{Y_{\max}}{M}$$

All scattered points lie within a domain of size 1×1. The goal is to compute interpolated values $F(X_i, Y_j)$ on the structured grid using linear interpolation.

3.2 Particle Update using Reverse Interpolation (Mover)

In addition to mapping particle values to the grid, the workflow introduces a reverse interpolation step (mover). After the grid values are calculated and normalized to a fixed range, they influence the motion of particles. For a particle located at (x_i, y_i), the field value is gathered using the same four neighboring grid points and identical weights:

$$F_i = w_{i,j}F(X_i, Y_j) + w_{i+1,j}F(X_{i+1}, Y_j) + w_{i,j+1}F(X_i, Y_{j+1}) + w_{i+1,j+1}F(X_{i+1}, Y_{j+1})$$

The particle position is then updated as:

$$x_i^{new} = x_i + F_i \cdot \Delta x$$

$$y_i^{new} = y_i + F_i \cdot \Delta y$$

If a particle moves outside the domain, it is marked as inactive and excluded from further computations.

3.3 Parallelization Strategies:

- Domain decomposition vs. Particle decomposition
- Efficient handling of race conditions during the scatter phase (e.g., atomic operations vs privatization)
- Reduction-based approaches
- Memory access patterns for both grid and particle data (read-heavy vs write-heavy loops)
- Proper load balancing as particle distribution evolves

4 Benchmarking Methodology

Input Data Configurations

Input data is generated using `input_fileMaker.c` with the following configurations:

(a) $N_x = 250$, $N_y = 100$, Number of points $= 0.9 \times 10^6$, Maxiter $= 10$

(b) $N_x = 250$, $N_y = 100$, Number of points $= 5 \times 10^6$, Maxiter $= 10$

(c) $N_x = 500$, $N_y = 200$, Number of points $= 3.6 \times 10^6$, Maxiter $= 10$

(d) $N_x = 500$, $N_y = 200$, Number of points $= 20 \times 10^6$, Maxiter $= 10$

(e) $N_x = 1000$, $N_y = 400$, Number of points $= 14 \times 10^6$, Maxiter $= 10$

5 Graphical Results

5.1 Assignment 06: Scatter Interpolation Results

5.1.1 Speedup vs Cores

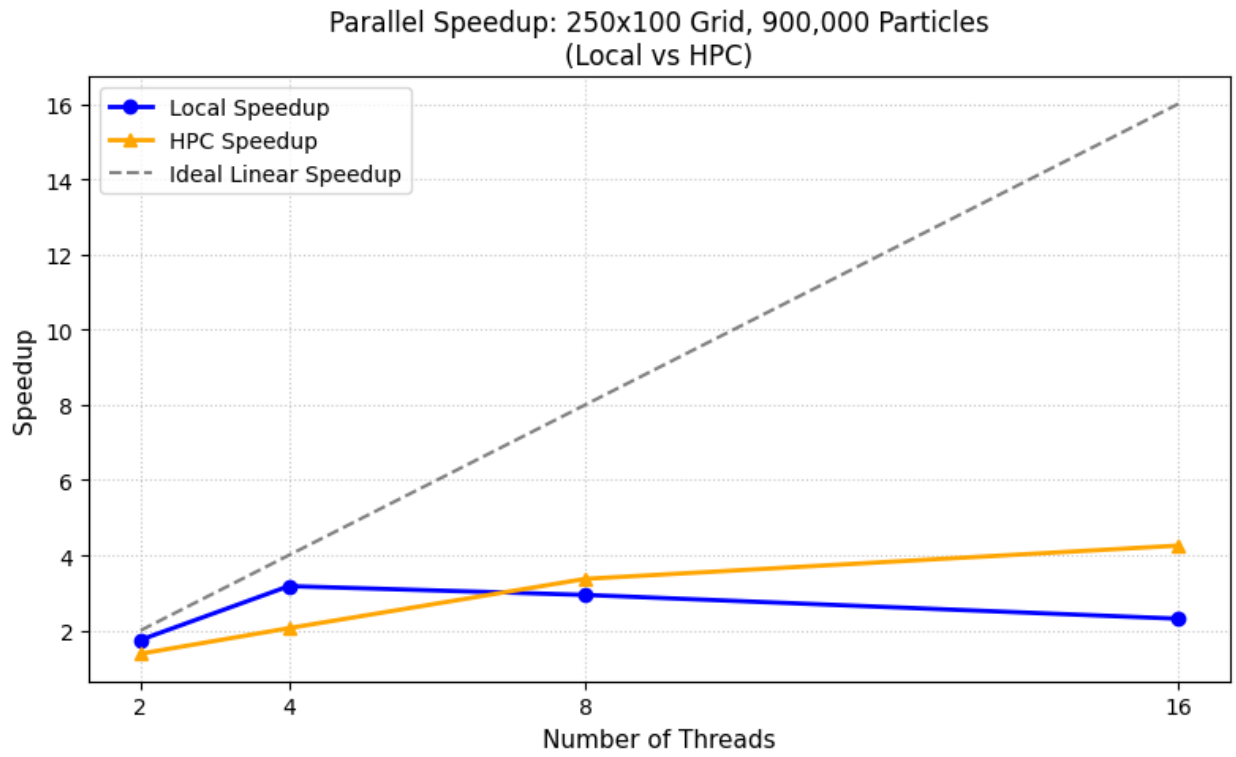


Figure 1: A6: Speedup vs Number of Cores - Config 01

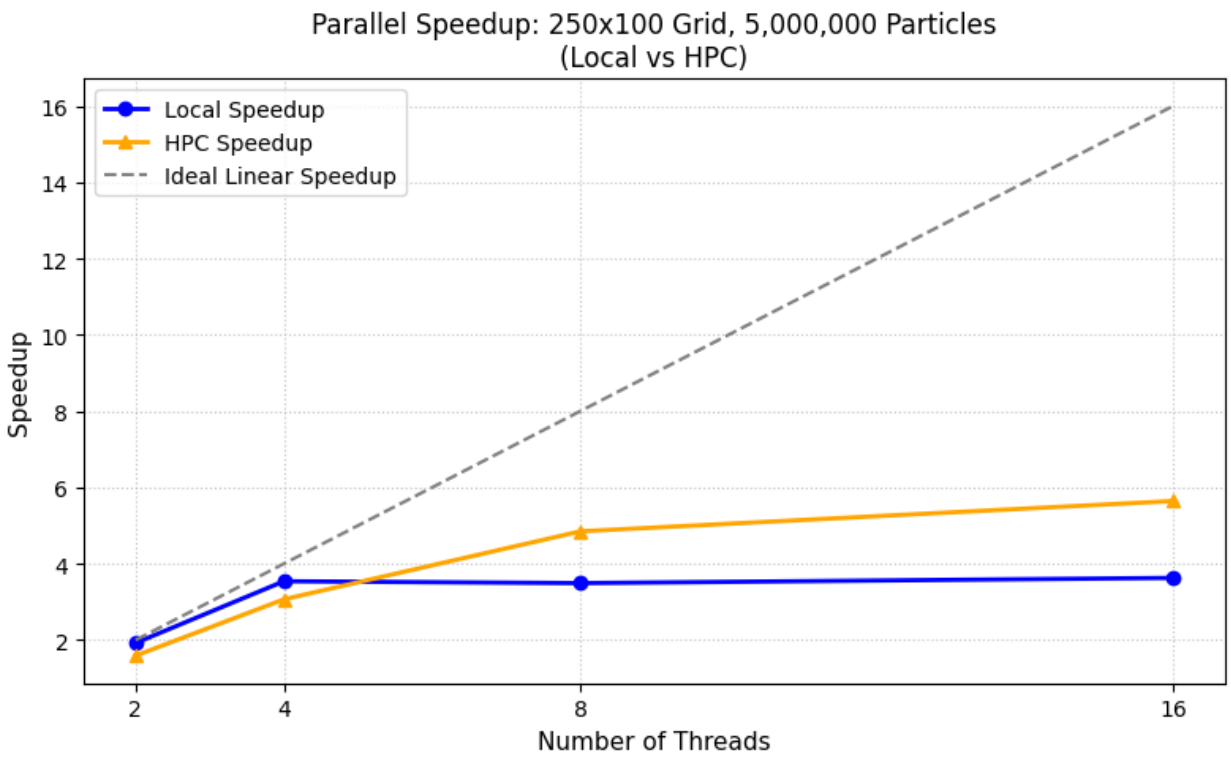


Figure 2: A6: Speedup vs Number of Cores - Config 02

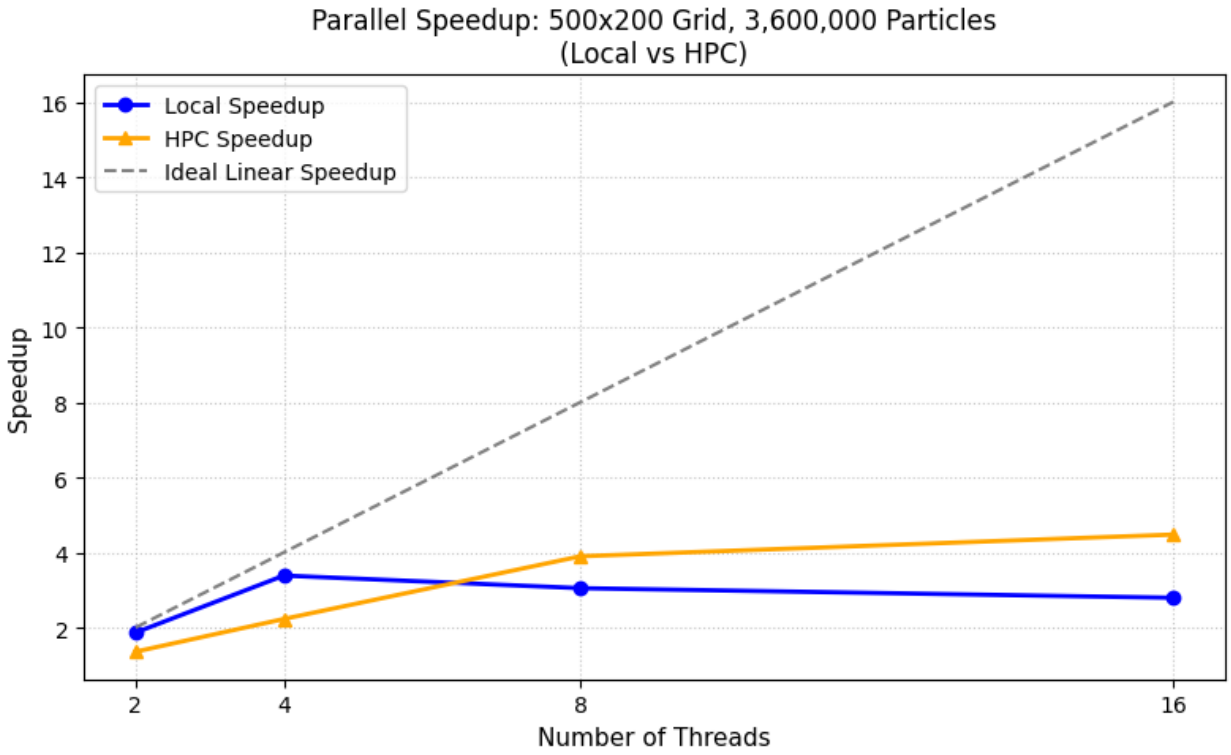


Figure 3: A6: Speedup vs Number of Cores - Config 03

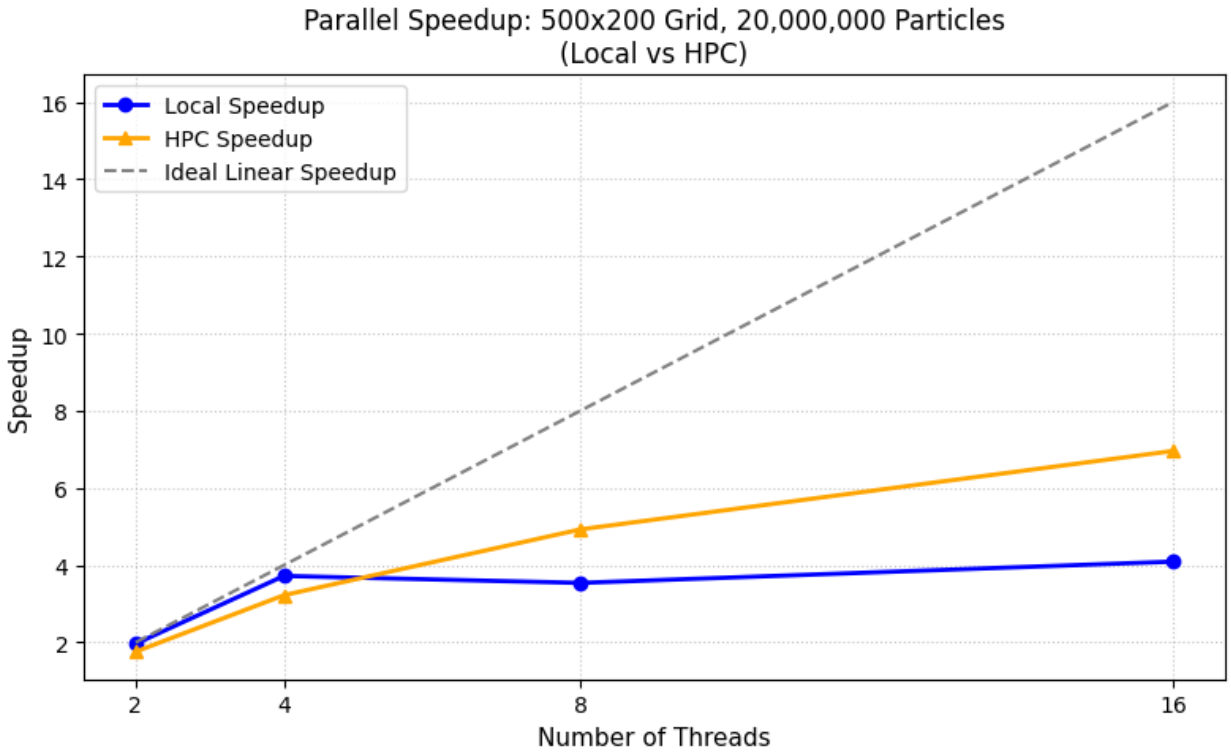


Figure 4: A6: Speedup vs Number of Cores - Config 04

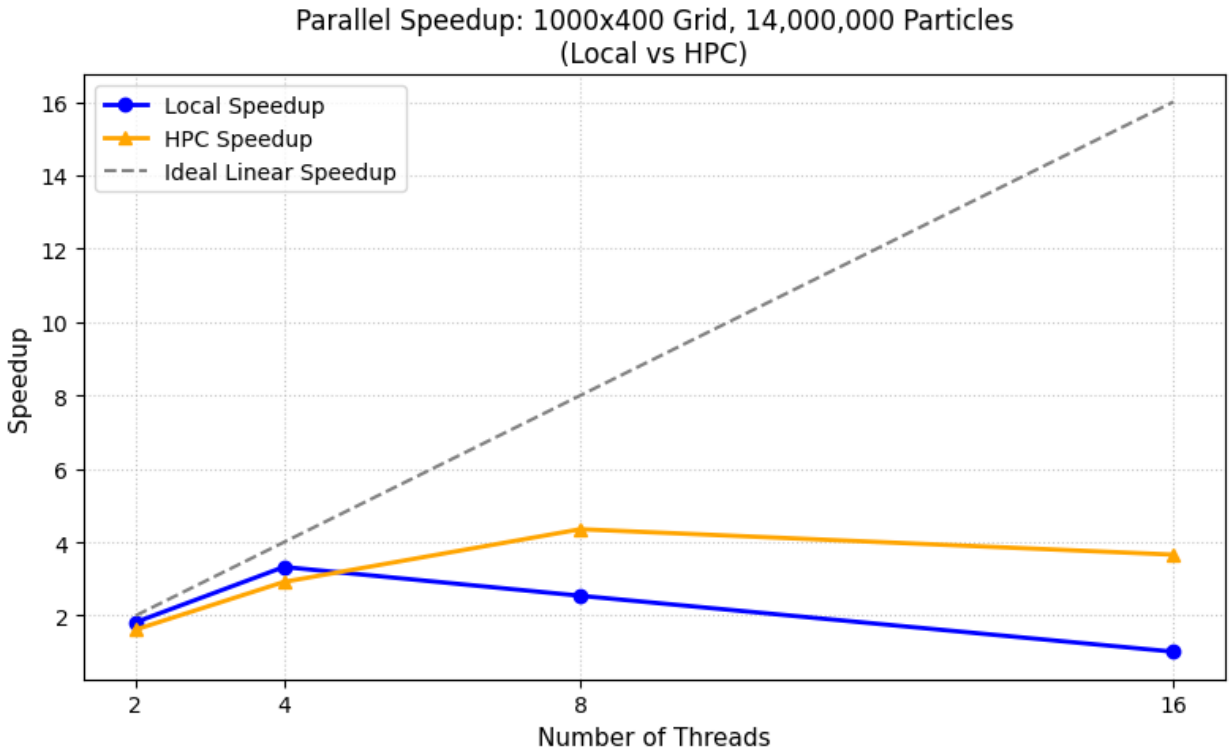


Figure 5: A6: Speedup vs Number of Cores - Config 05

5.1.2 Execution time vs Cores

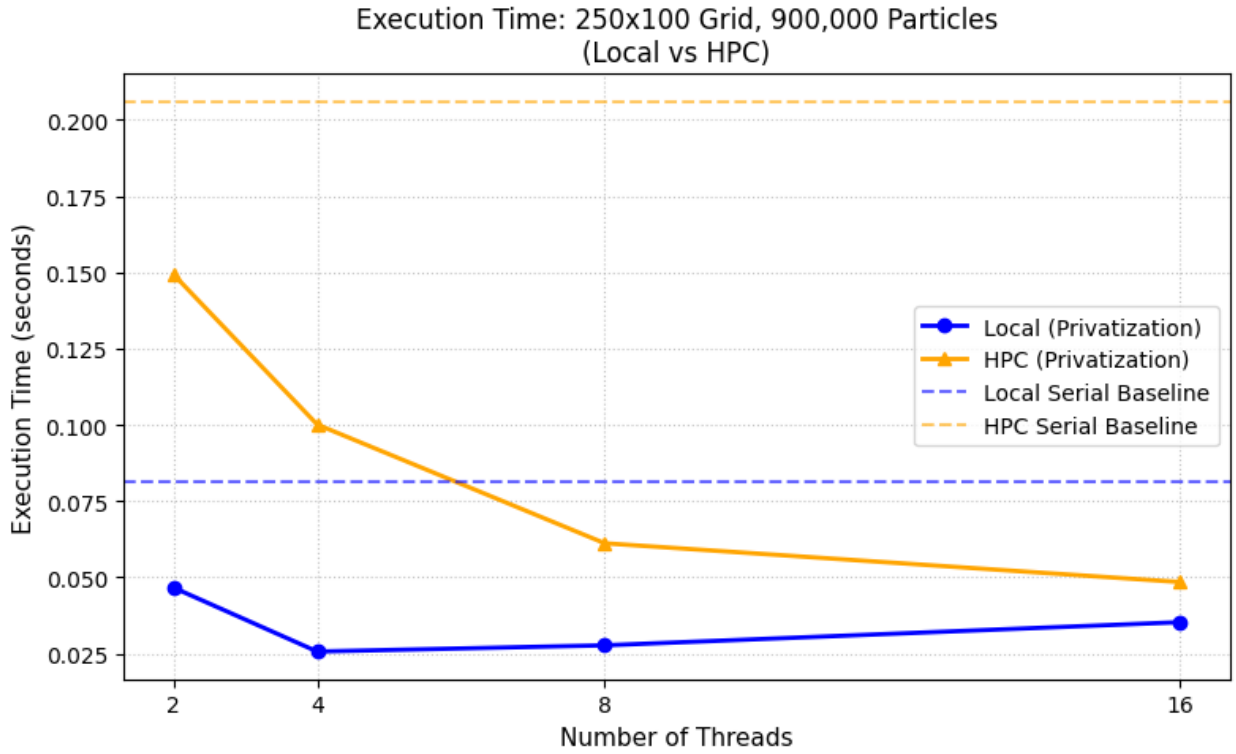


Figure 6: A6: Execution Time vs Number of Cores - Config 01

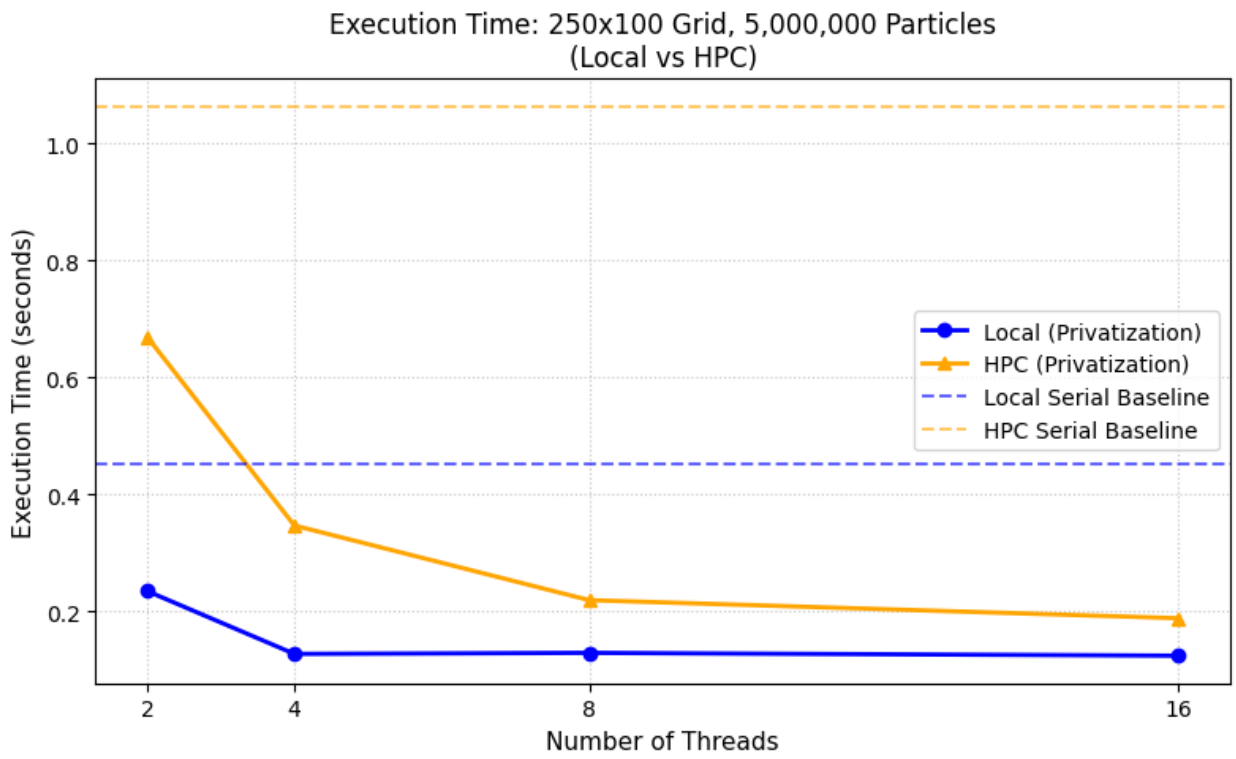


Figure 7: A6: Execution Time vs Number of Cores - Config 02

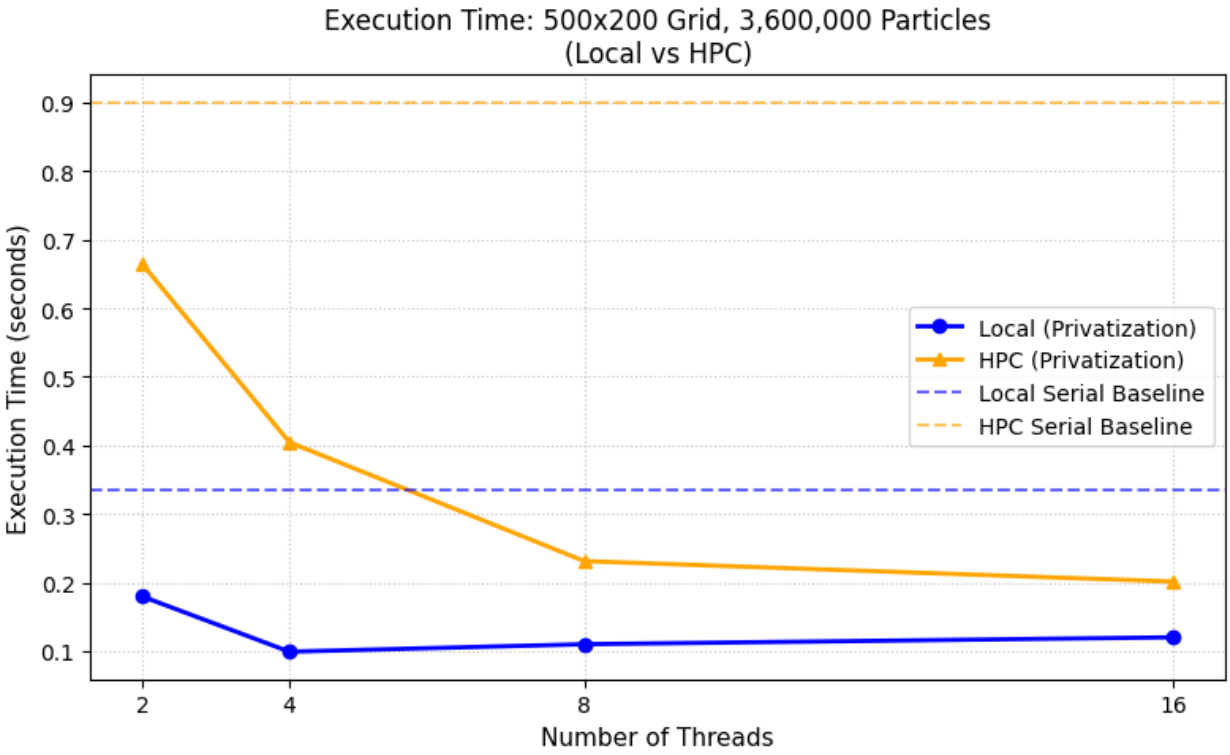


Figure 8: A6: Execution Time vs Number of Cores - Config 03

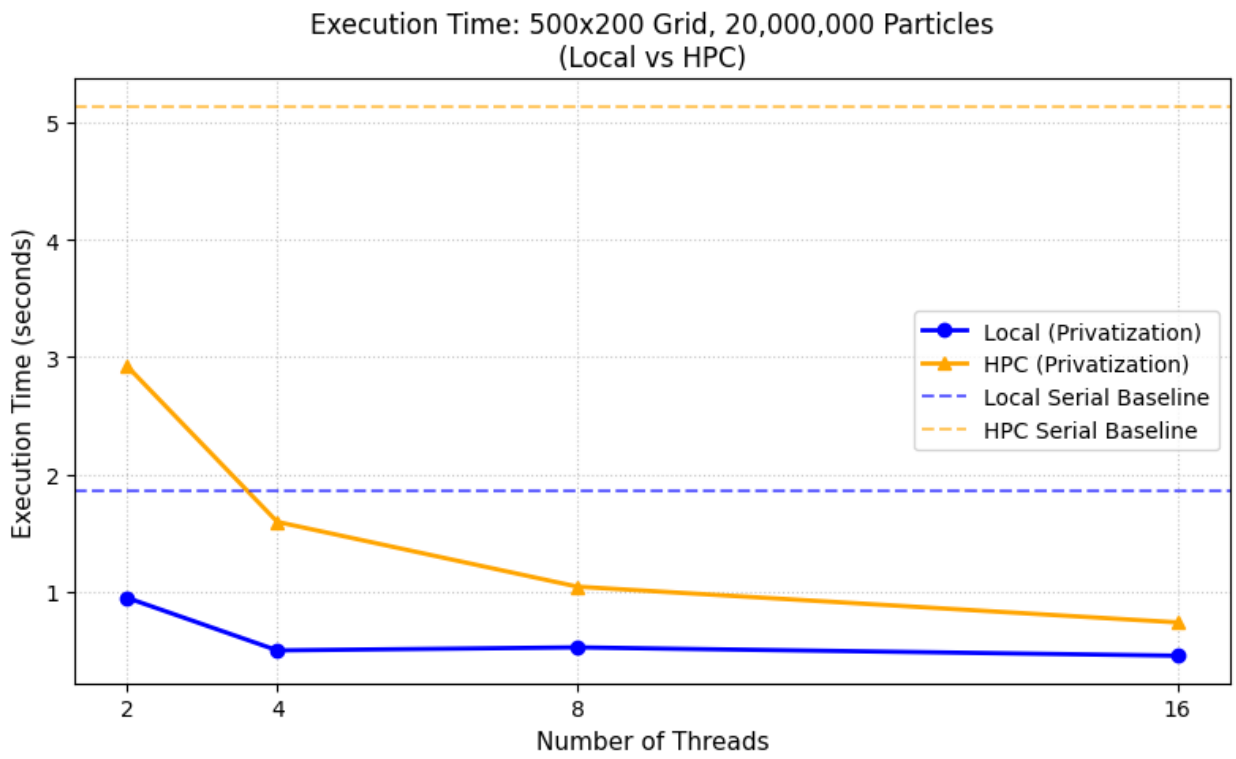


Figure 9: A6: Execution Time vs Number of Cores - Config 04

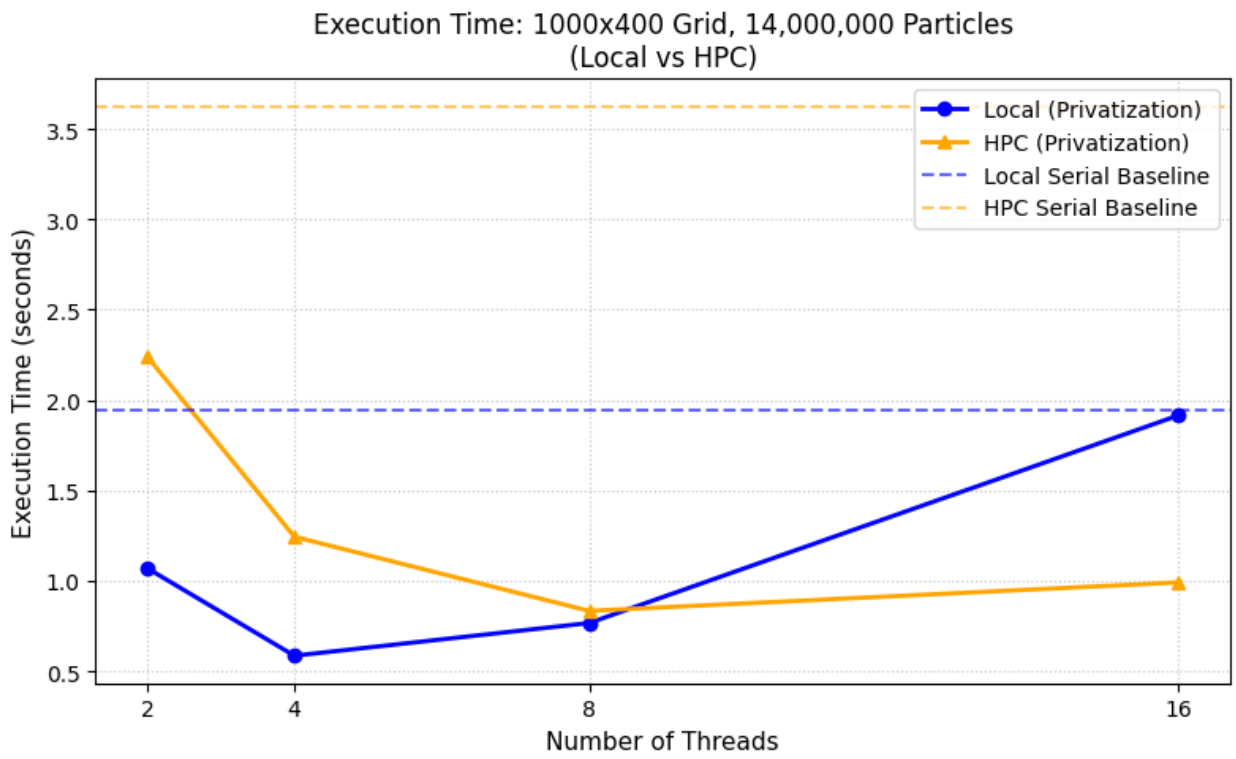


Figure 10: A6: Execution Time vs Number of Cores - Config 05

5.2 Assignment 07: Particle Mover Results

5.2.1 Speedup vs Cores

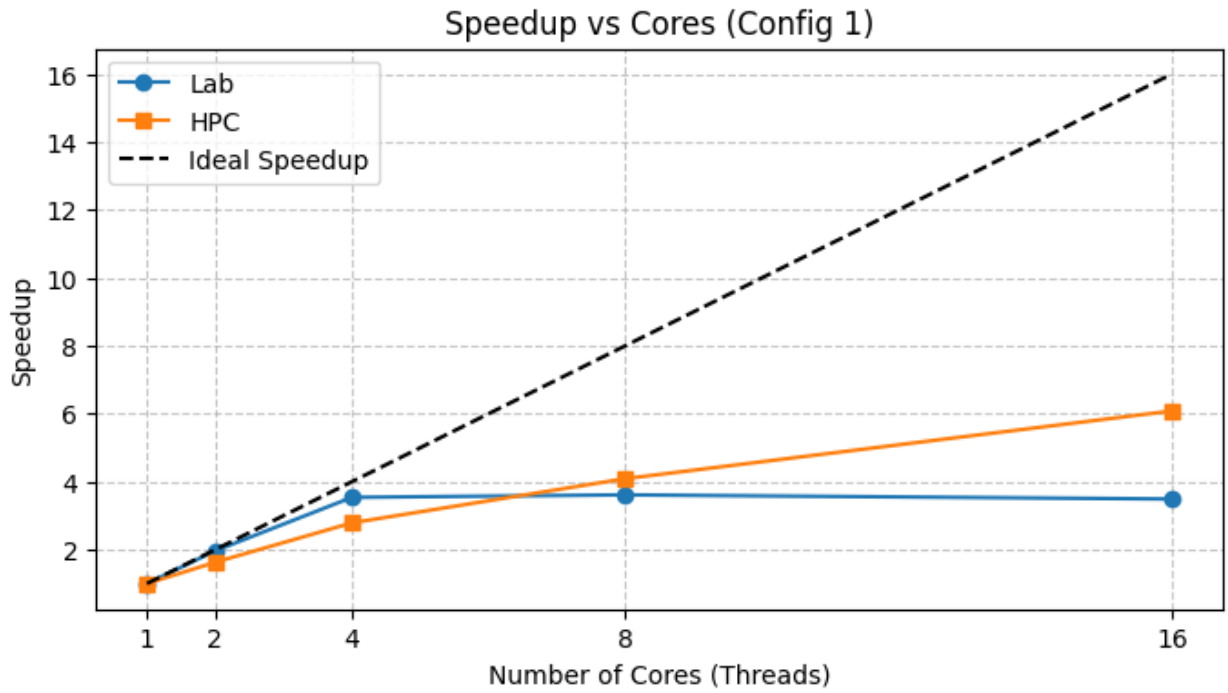


Figure 11: A7: Speedup vs Number of Cores - Config 01

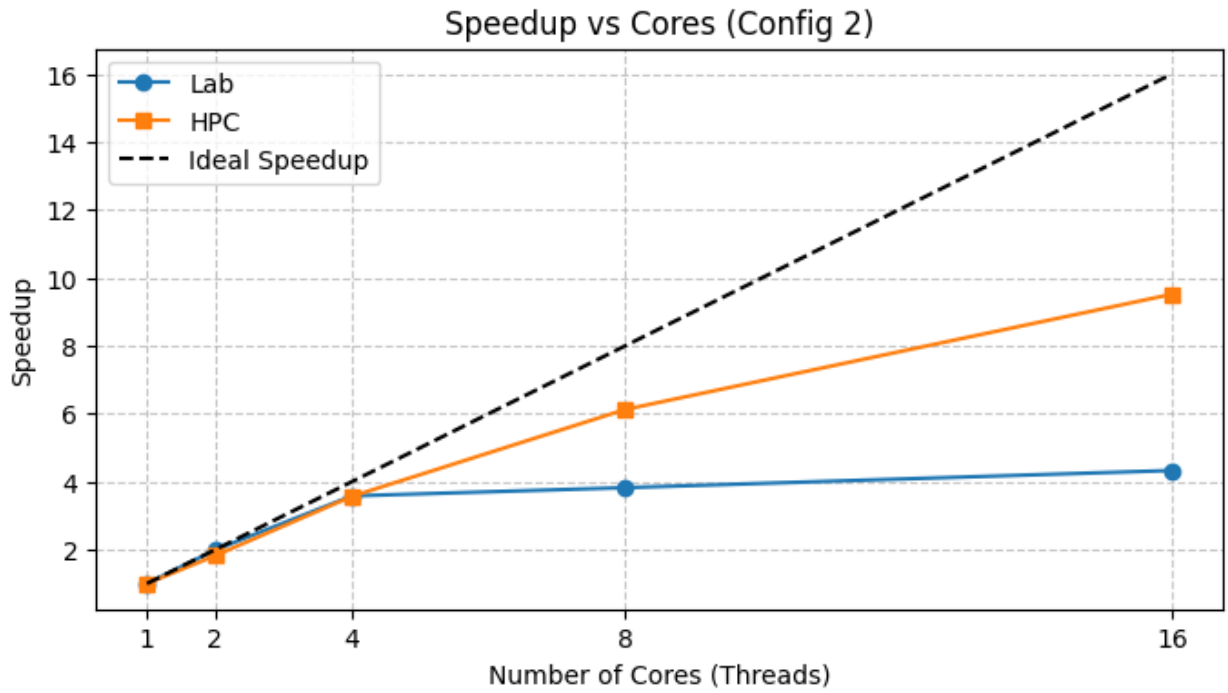


Figure 12: A7: Speedup vs Number of Cores - Config 02

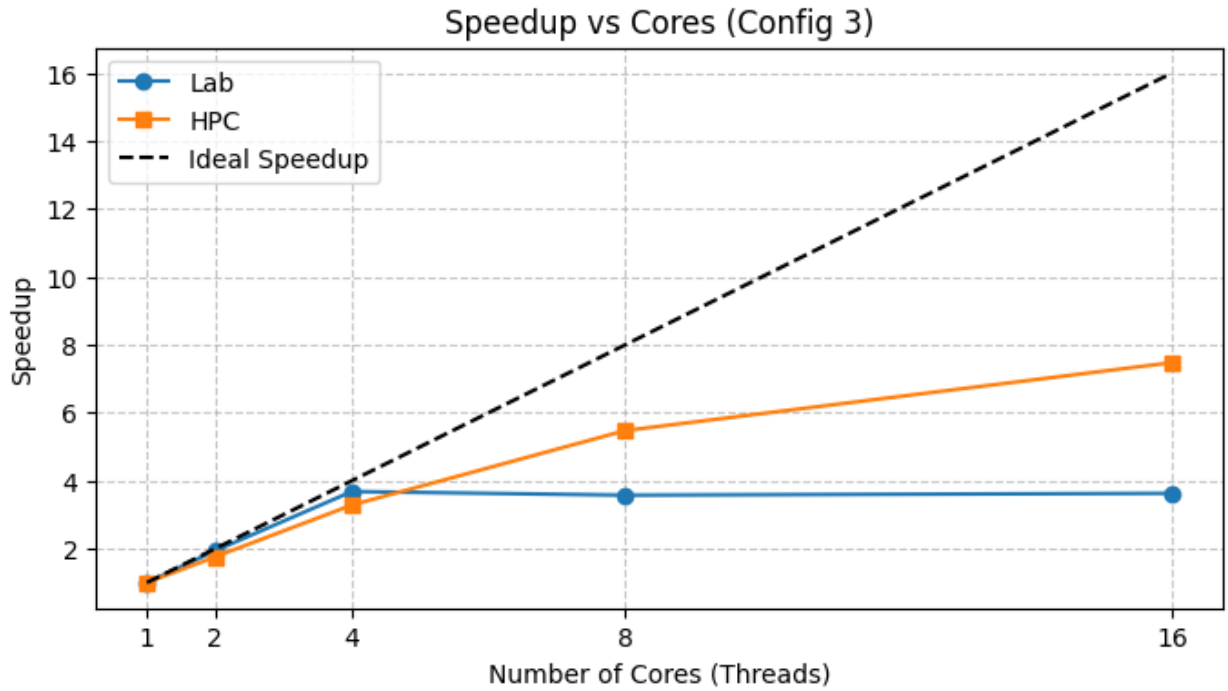


Figure 13: A7: Speedup vs Number of Cores - Config 03

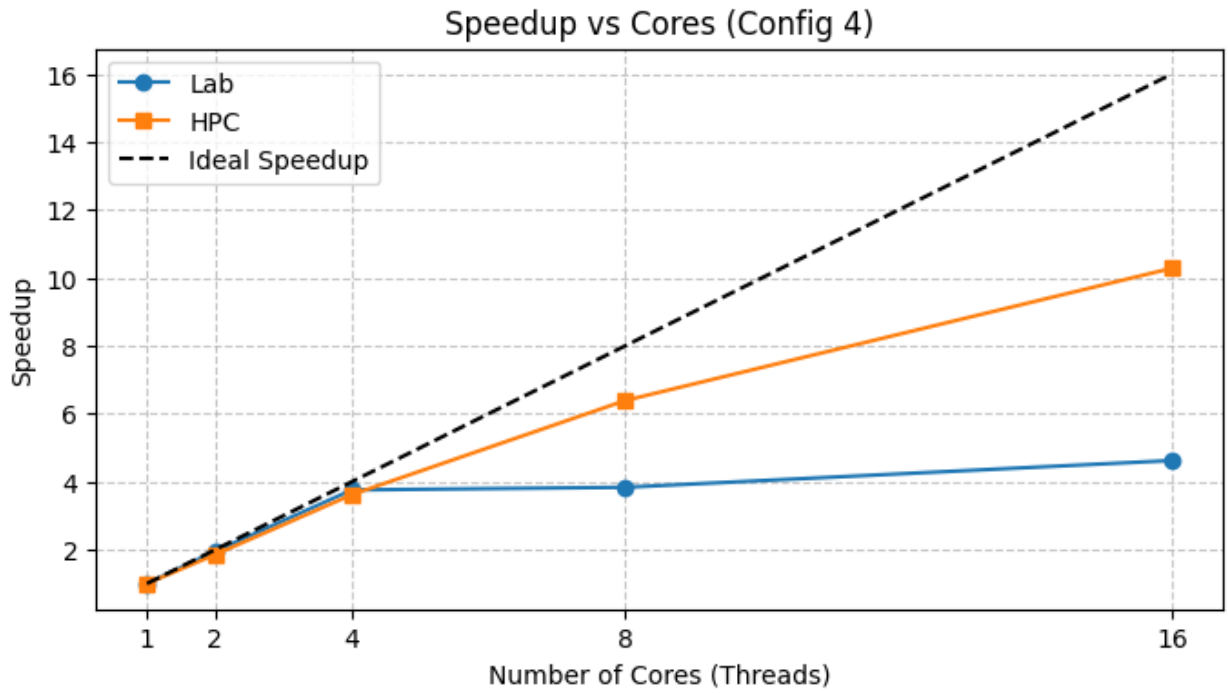


Figure 14: A7: Speedup vs Number of Cores - Config 04

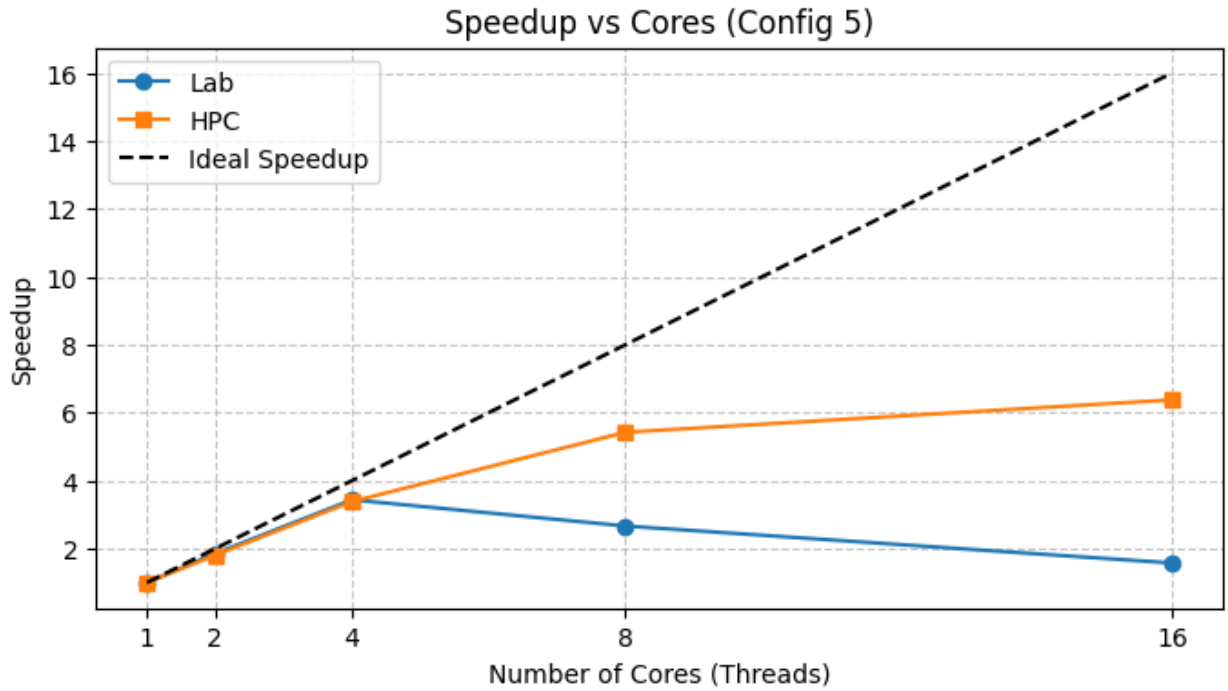


Figure 15: A7: Speedup vs Number of Cores - Config 05

5.2.2 Execution time vs Cores

Figure 16: A7: Execution Time vs Number of Cores - Config 01

Figure 17: A7: Execution Time vs Number of Cores - Config 02

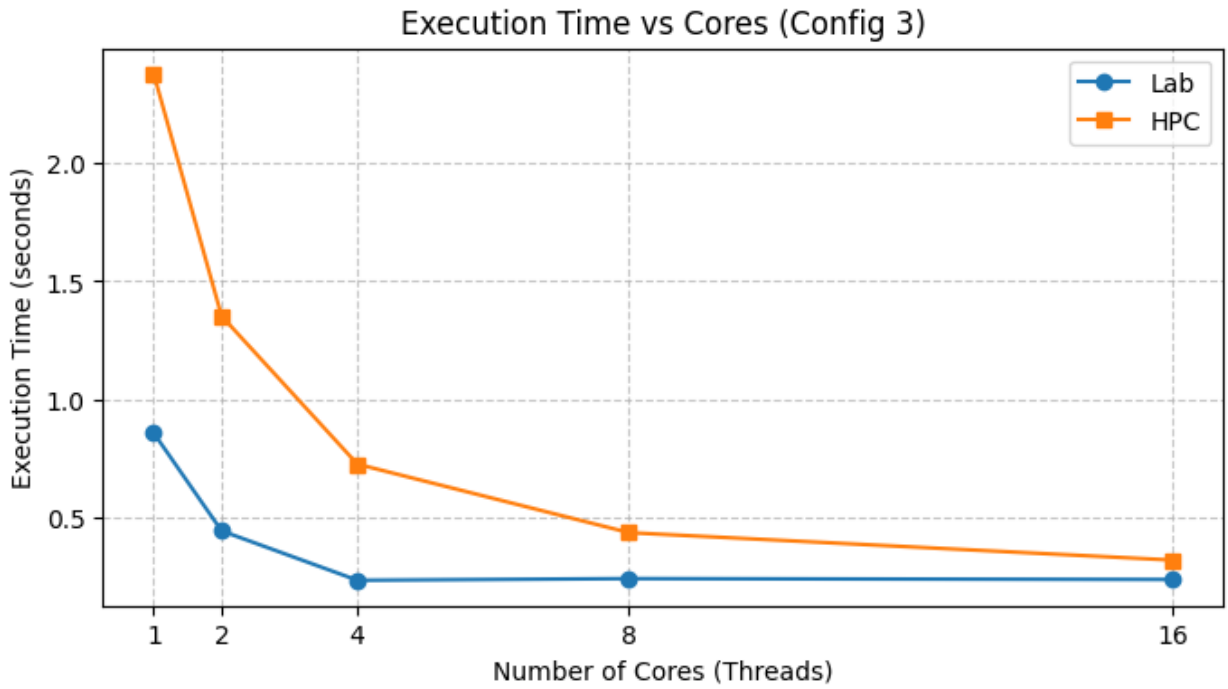


Figure 18: A7: Execution Time vs Number of Cores - Config 03

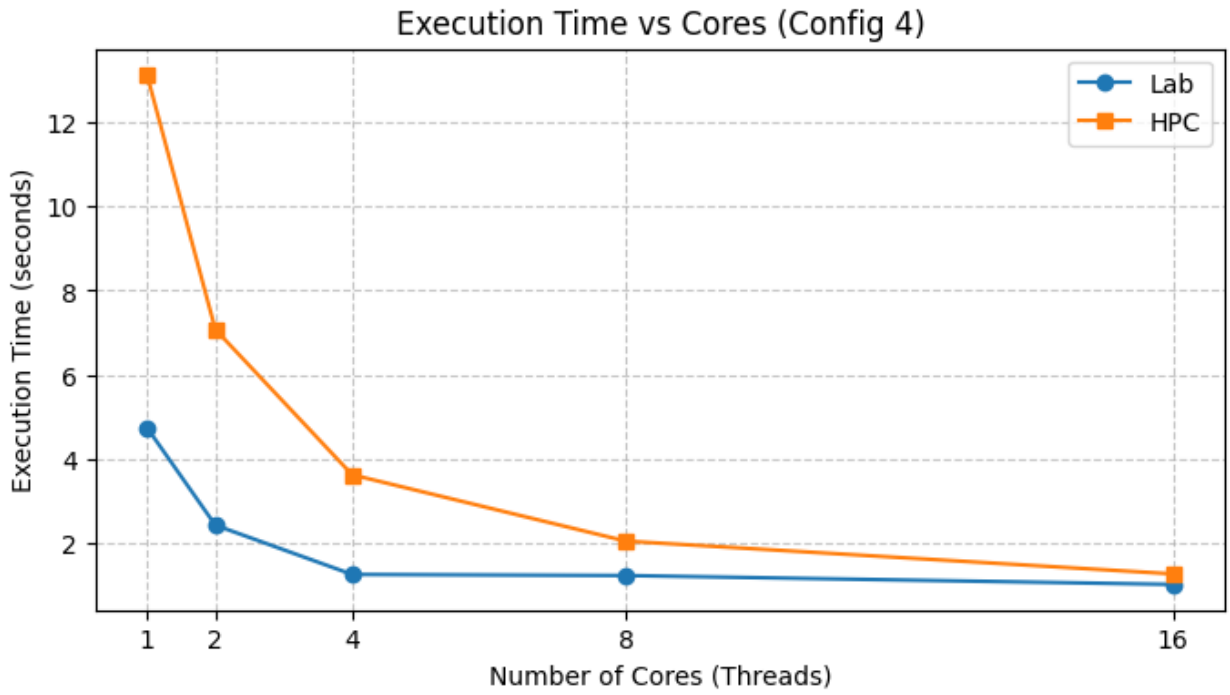


Figure 19: A7: Execution Time vs Number of Cores - Config 04

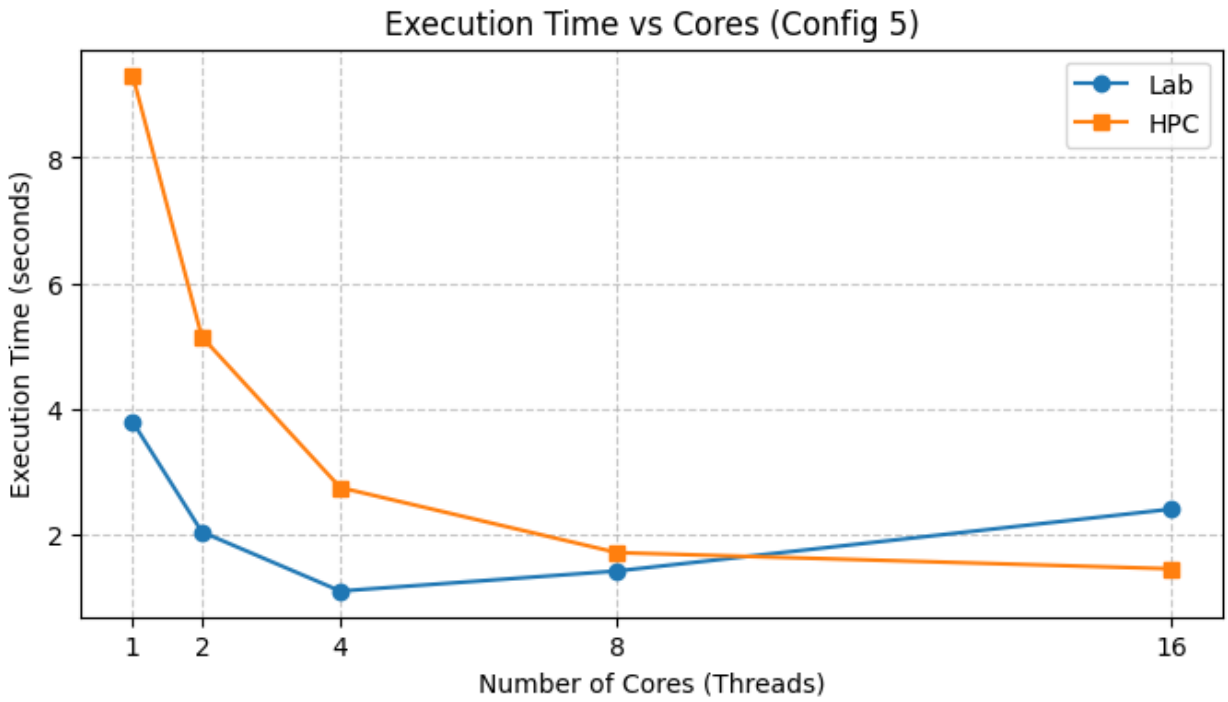


Figure 20: A7: Execution Time vs Number of Cores - Config 05

6 Analysis and Questions

6.1 Assignment 06 Analysis (Scatter Interpolation)

Q1. **Write the pseudo-code of your parallel implementation in the report along with an illustrative diagram of your implementation.**

(i) Pseudocode: Parallel Interpolation using *Privatization*

Algorithm 1 Parallel Scatter Interpolation using Privatization

---

Input: mesh_value array, points array
Output: Updated mesh_value array
Compute $inv_dx \leftarrow 1/dx$, $inv_dy \leftarrow 1/dy$

Parallel Region Start
Each thread allocates $local_mesh[GRID_X \times GRID_Y] \leftarrow 0$
for *each point* $p = 1$ *to* num_points **do in parallel**
  Read (x, y) from points[p], set $f \leftarrow 1.0$
  $i \leftarrow \lfloor x \cdot inv_dx \rfloor, \quad j \leftarrow \lfloor y \cdot inv_dy \rfloor$
  Clamp $i \in [0, NX - 1], \quad j \in [0, NY - 1]$
  $X_i \leftarrow i \cdot dx, \quad Y_j \leftarrow j \cdot dy$
  $l_x \leftarrow x - X_i, \quad l_y \leftarrow y - Y_j$
  Compute weights:

$$w_{00} = (dx - l_x)(dy - l_y), \quad w_{10} = l_x(dy - l_y)$$

$$w_{01} = (dx - l_x)l_y, \quad w_{11} = l_x l_y$$

  Compute indices:

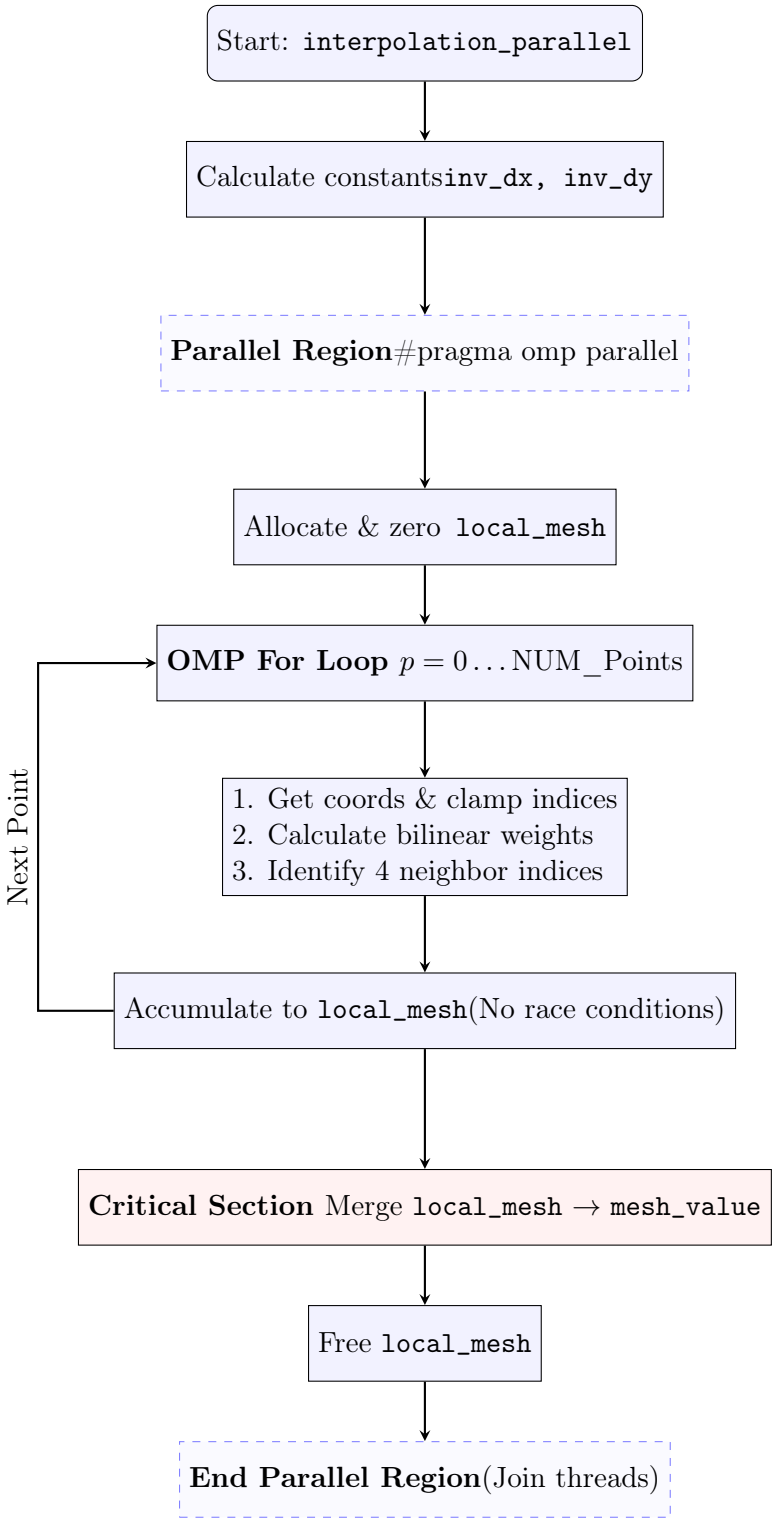
$$idx_{00},\ idx_{10},\ idx_{01},\ idx_{11}$$

  Accumulate into local mesh:

$$local_mesh[idx] += w \cdot f$$

Critical Section:
for *each grid index* k **do**
  $mesh_value[k] += local_mesh[k]$
Free $local_mesh$
Parallel Region End

(ii) Schematic Flow Diagram: Parallel Interpolation



Q2. **Explain your parallel implementation approach and how you handled race condi-**

tions in your code.

- **Privatization + Reduction-style Strategy:**
  (a) Each thread maintains its own local copy of the mesh (local_mesh) instead of directly updating the shared global mesh.
  (b) This allows threads to perform interpolation independently without any synchronization overhead during computation.
  (c) Once all threads finish processing, a reduction-style step is performed in which all local meshes are combined into the global mesh.
  (d) In this strategy, only the final merge requires synchronization.
- **Race Condition:**
  (a) Race condition may occur when multiple particles contribute to the same grid points, causing concurrent writes to shared mesh values.
  (b) This issue is avoided by using thread-local meshes, ensuring that no shared memory is modified during the parallel loop.
  (c) The only point of potential conflict is during the merge phase, which is safely handled using a **critical** section to enforce mutual exclusion and maintain correctness.

Q3. **Plot graphs for each of the five input configurations for speedup vs cores and execution time vs cores (increment cores starting from 2 to 16 by a factor of 2). Ensure no one else is using the machine while running the code to get good performance.**

Refer to the Graphical Results section above (Section 5.1).

Q4. **Compute the parallel efficiency and comment on the scalability of your implementation.**

Using the 500x200 grid (20M particles) at 16 threads:

- **Speedup (S):** $\frac{1.863}{0.450} \approx 4.14$
- **Parallel Efficiency (E):** $\frac{4.14}{16} \approx 25.8\%$

Scalability is sub-linear. Efficiency drops at higher thread counts because the final reduction step (merging massive thread-local grids) becomes a sequential memory bottleneck, strictly adhering to Amdahl's Law.

Q5. **Compare the performance of your parallel code with the serial version and report the maximum speedup achieved.**

The maximum speedup was observed on the HPC cluster for the configuration of 5,000,000 particles on a 250x100 grid.

- **Serial Time:** 1.064 seconds
- **Privatization (16 Threads):** 0.188 seconds
- **Maximum Speedup: 5.66x**

Q6. **Try to explain the speedup achieved and all the observations using your current HPC knowledge.**

- **Privatization:** Best performance due to lock-free accumulation in thread-local L1/L2 caches, completely avoiding race conditions during the massive particle loop.
- **Atomics:** Poor speedup ($\sim 1.5x$). Locking shared grid cells 20 million times creates massive cache coherency overhead and stalls the CPU pipeline.
- **Domain Decomposition:** Worse than the serial baseline. This points to a severe load imbalance caused by uneven particle distribution, leaving some threads idle while others overload.

Q7. **If given unlimited HPC resources, how would you further optimize the implementation?**

I would port the CPU-bound OpenMP implementation to a massively parallel GPU architecture using CUDA to leverage thousands of cores. To overcome single-node memory limits, I would combine this with MPI to partition the global grid across a supercomputing cluster, using explicit message passing for halo exchanges.

Q8. **Discuss any load imbalance or performance bottlenecks observed in your implementation.**

- Performance Bottleneck (The Reduction Phase): As thread counts and grid sizes grow, the sequential #pragma omp critical block becomes a massive bottleneck. One thread merges its grid while 15 others sit idle, wasting CPU cycles.
- Memory Bandwidth Bound: Processing 20 million particles across 16 threads simultaneously exhausts the memory bandwidth controller of a standard CPU. Threads end up stalling while waiting for data fetches from main RAM.
- The #pragma omp for directive splits the particle array uniformly. If particles happen to be sorted such that one chunk represents a dense cluster of points and another chunk represents empty space, cache locality will differ heavily between threads, causing minor execution time skew (load imbalance).

Q9. **Suggest at least one alternative approach (data structure or parallelization method) to improve performance.**

Alternative Method: Spatial Sorting (Particle Binning) Instead of processing particles in their random memory order, first do a lightweight parallel sort (or binning) of the particles based on the grid cell they belong to. Once particles are sorted spatially, threads can process chunks of the array with massive cache-hit rates, because consecutive particles will be updating the exact same grid cells in memory rather than causing random memory jumps.

Q10. **You are required to come up with different data structures or OpenMP-based approaches to increase the performance of the code. Anyone coming up with good results may get bonus marks for this assignment. Leader board will be based on performance and efficiency**

To secure performance beyond standard privatization, a **Grid Coloring** (e.g., Red-Black) approach can be implemented. Grid cells are colored so no adjacent cells share the same color. Threads process particles phase-by-phase (e.g., updating only "Red" cells first). This guarantees no simultaneous writes to adjacent nodes, allowing completely lock-free `#pragma`

`omp for` loops. This eliminates both the heavy memory footprint of Privatization and the massive synchronization overhead of Atomics.

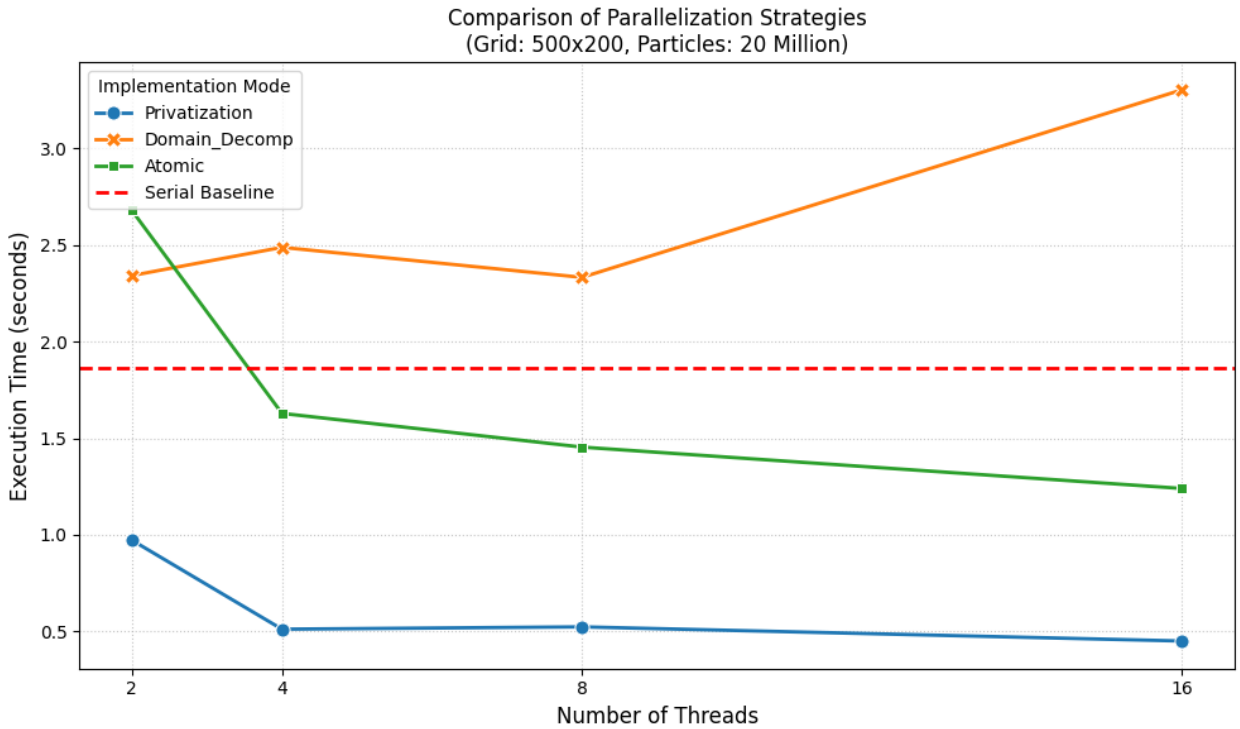


Figure 21: Comparison of Parallelization Strategies (Privatization vs Domain Decomposition vs Atomics)

6.2 Assignment 07 Analysis (Particle Mover)

Q1. **Write the pseudocode of your parallel implementation in the report along with an illustrative diagram of your implementation.**

(i) Pseudocode: Parallel Mover using *Particle Decomposition*

Algorithm 2 Parallel Gather (Mover) using Particle Decomposition

Input: mesh_value array, points array
Output: Updated particle positions

Parallel Region Start
Distribute iterations of particles across threads using OpenMP (`#pragma omp parallel for`)
for *each particle* $p = 1$ *to* NUM_Points *(assigned to different threads)* **do in parallel**
  if *points[p].is_void* **then**
    continue
  Read (x, y) from points[p]
  $i \leftarrow \lfloor x/dx \rfloor, \quad j \leftarrow \lfloor y/dy \rfloor$
  if $i < 0$ *or* $i \geq NX$ *or* $j < 0$ *or* $j \geq NY$ **then**
    continue
  $l_x \leftarrow x - (i \cdot dx), \quad l_y \leftarrow y - (j \cdot dy)$
  Compute weights:

$$w_{00} = (dx - l_x)(dy - l_y), \quad w_{10} = l_x(dy - l_y)$$

$$w_{01} = (dx - l_x)l_y, \quad w_{11} = l_x l_y$$

  Gather grid values (read-only access):

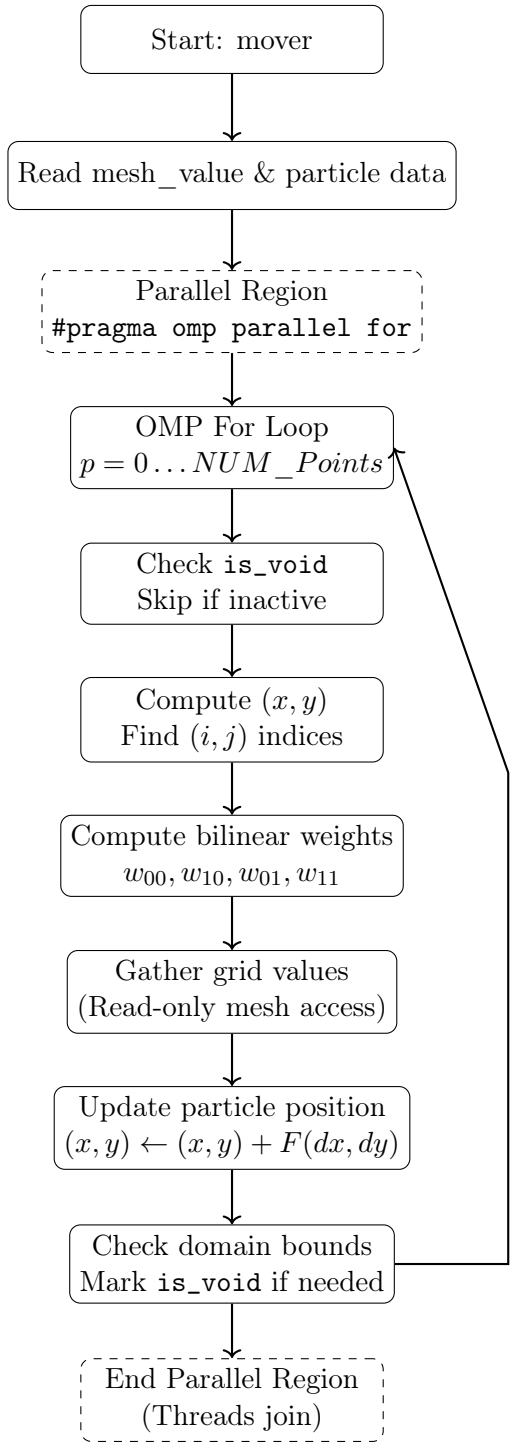
$$F = w_{00} \cdot mesh[i, j] + w_{10} \cdot mesh[i + 1, j] + w_{01} \cdot mesh[i, j + 1] + w_{11} \cdot mesh[i + 1, j + 1]$$

  Update particle position (thread-private data):

$$x \leftarrow x + F \cdot dx, \quad y \leftarrow y + F \cdot dy$$

  Write back to points[p]
  if $x < 0$ *or* $x > 1$ *or* $y < 0$ *or* $y > 1$ **then**
    points[p].is_void ← true

(ii) Schematic Flow Diagram: Parallel Mover



Note: Each thread works on a distinct subset of particles, with no shared writes. Parallel Region End

Q2. **Explain your parallel implementation approach and how you handled race conditions in your code.**

- **Particle Decomposition Strategy:**
  (a) The mover phase is parallelized using **particle decomposition**, where the loop over particles is distributed across threads using `#pragma omp parallel for`.
  (b) Each thread processes a subset of particles independently, performing grid-to-particle interpolation and updating particle positions.
  (c) The grid (`mesh_value`) is accessed in a read-only manner, avoiding any need for synchronization during computation.
  (d) This approach ensures good load balancing and minimal overhead, as the computation is embarrassingly parallel.
- **Race Condition:**
  (a) Race conditions typically arise when multiple threads write to shared memory locations simultaneously.
  (b) In the mover phase, this issue does not occur because each thread updates only its own particle data (`points[p]`), with no shared writes.
  (c) The shared grid is only read and never modified during this phase, eliminating the possibility of write conflicts.
  (d) Therefore, no synchronization mechanisms such as atomics, locks, or critical sections are required, ensuring both correctness and high performance.

Q3. **Plot graphs for each of the five input configurations for speedup vs cores and execution time vs cores (increment cores starting from 2 to 16 by a factor of 2).**

Refer to the Graphical Results section above (Section 5.2).

Q4. **Compute the parallel efficiency and comment on the scalability of your implementation.**

Parallel efficiency ($E = S/p$) ranges from 0.10 to 1.00 on the Lab system, and 0.38 to 0.93 on the HPC. The implementation shows **sub-linear scalability**, as efficiency drops at higher thread counts due to synchronization overhead and memory bandwidth limits.

Q5. **Compare the performance of your parallel code with the serial version and report the maximum speedup achieved.**

The parallel version significantly outperforms the serial baseline. The maximum speedups achieved are:

- **Lab:** 4.62× (Config 4, 16 threads)
- **HPC:** 10.28× (Config 4, 16 threads)

Q6. **Try to explain the speedup achieved and all the observations using your current HPC knowledge.**

The speedup is sub-linear because the workload is highly memory-bound. The interpolation phase suffers from thread contention due to atomic operations used to prevent race conditions. The HPC system achieves a higher speedup (10.28× vs 4.62×) due to its vastly superior memory bandwidth, larger caches, and better hardware support for atomics.

Q7. **If given unlimited HPC resources, how would you further optimize the implementation?**

(a) **Distributed Memory:** Use MPI with domain decomposition to scale across multiple compute nodes.

(b) **GPU Acceleration:** Port the highly parallel particle updates to GPUs using CUDA or OpenACC.

(c) **Data Locality:** Periodically sort particles by spatial coordinates (e.g., Morton codes) to minimize cache misses and thread contention.

Q8. **Discuss any load imbalance or performance bottlenecks observed in your implementation.**

- Load Imbalance due to Inactive Particles: As iterations progress, some particles move outside the domain and are marked as `is_void`. Threads assigned more inactive particles perform less useful work, leading to slight load imbalance despite static scheduling.
- Memory Access Bottleneck: The mover operation involves frequent reads from the shared grid (`mesh_value`) at non-contiguous locations, which can lead to poor cache utilization and increased memory latency, especially for large grids.
- Branch Divergence and Conditional Overhead: The presence of conditional checks (e.g., `is_void` and boundary checks for `i, j`) inside the parallel loop can introduce branch divergence among threads, slightly affecting performance efficiency.

Q9. **Suggest at least one alternative approach (data structure or parallelization method) to improve performance.**

An alternative approach to improve performance is to use **dynamic scheduling** in OpenMP (e.g., `#pragma omp parallel for schedule(dynamic, chunk_size)`). Since the number of active particles varies over time due to particles becoming `is_void`, dynamic scheduling helps distribute work more evenly among threads, reducing load imbalance. Additionally, reorganizing particle data into a **Structure of Arrays (SoA)** format can improve cache locality and memory access efficiency, further enhancing performance.

Q10. **You are required to come up with different data structures or OpenMP-based approaches to increase the performance of the code.**

- **Avoided Domain Decomposition:** Domain-based approaches were not used because particle distribution is highly non-uniform and dynamic (due to movement and `is_void`), which can lead to significant load imbalance and additional overhead in mapping particles to grid regions.
- **Avoided Atomics/Privatization:** Synchronization-based methods such as atomics or privatization were not used since the mover phase involves only read operations on the grid and independent updates to particles, making such techniques unnecessary and adding avoidable overhead.
- **Chosen Approach (Particle Decomposition):** Particle decomposition was selected as it naturally fits the mover operation, enabling independent per-particle computation with no race conditions, minimal synchronization, and good scalability.

Q11. **Analyze the performance of interpolation and mover phases separately. Which phase is the bottleneck and why?**

The **Mover phase** is the slight bottleneck (e.g., HPC average 1.27s vs Interpolation 1.14s).

- **Interpolation:** Bottlenecked by **write contention** (overhead from atomic operations to resolve race conditions).
- **Mover:** Bottlenecked by **memory latency**. Reading grid values based on unorganized particle positions causes highly irregular memory access, leading to severe cache thrashing.

7 Conclusions

- **Dynamic operations impact:** Insertion and deletion introduce overhead and reduce scalability compared to the static Mover, making the algorithm more memory-bound.
- **Phase bottlenecks:** The interpolation phase (scatter) is primarily limited by write contention and atomic operation overhead, whereas the mover phase (gather) is bottlenecked by memory latency due to irregular, non-contiguous cache accesses.
- **Parallel performance:** OpenMP achieves significant speedup (up to 10.28× on the HPC cluster), but scalability remains sub-linear due to memory bandwidth saturation and synchronization costs at higher thread counts.
- **Approach comparison:** Immediate replacement is simpler and more scalable, whereas deferred insertion adds compaction overhead despite better control.
- **Architectural influence:** The HPC cluster significantly outperforms the Lab PC in parallel execution, demonstrating that server-grade memory bandwidth and larger caches are critical for handling memory-bound, scatter-gather workloads.
- **Future optimizations:** To overcome the current shared-memory bottlenecks, future improvements should focus on spatial sorting of particles to enhance data locality, or leveraging GPU architectures to handle the massively parallel, independent particle updates.